

The background of the entire image is a dark blue field filled with a pattern of red dots. These dots are arranged in a way that they form a large, faint, stylized circular shape in the center, with the density of the dots increasing towards the center.

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

GIẢI THUẬT VÀ CÀI ĐẶT CHƯƠNG TRÌNH SONG SONG BÀI TOÁN CÓ SỰ PHỤ THUỘC DỮ LIỆU

Lattice Boltzmann Method (D2Q9)

Người thực hiện:

Sinh viên: Krizpham

175408 - **ONE LOVE ONE FUTURE** - Tính toán Hiệu năng Cao

Nội dung trình bày I

1. Mô hình Toán học và Phương pháp Giải
2. Sự Phụ Thuộc Dữ Liệu
3. Đại Lượng Vĩ Mô và Ký Hiệu
4. Cài Đặt Thực Tế
5. Giải Thuật Song Song SPMD
6. Chi Tiết Truyền Thông
7. Kết Quả Thực Nghiệm
8. Kết Luận



Phương trình LBE (Lattice Boltzmann Equation):

$$f_i(x + c_i \Delta t, t + \Delta t) = f_i(x, t) - \omega[f_i(x, t) - f_i^{eq}(x, t)] \quad (1)$$

Công thức giải:

► Khởi tạo: $f_i(x, 0) = f_i^{eq}(\rho = 1, u = 0)$

► Tại bước thời gian $n + 1$:

1. Collision (Va chạm):

$$f_i^*(x, t) = f_i(x, t) - \omega(f_i(x, t) - f_i^{eq}(x, t))$$

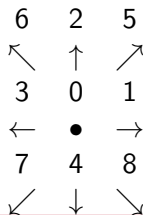
2. Streaming (Di chuyển):

$$f_i(x + c_i \Delta t, t + \Delta t) = f_i^*(x, t)$$

Mô hình D2Q9:

- ▶ **D2:** 2 chiều (2D)
- ▶ **Q9:** 9 hướng vận tốc
- ▶ Vector vận tốc: $c_i = (c_{ix}, c_{iy})$
- ▶ Trọng số: w_i

Sơ đồ hướng:



Trọng số và Vector:

- ▶ $w_0 = \frac{4}{9}$ (center)
- ▶ $w_{1-4} = \frac{1}{9}$ (cardinal)
- ▶ $w_{5-8} = \frac{1}{36}$ (diagonal)

Vector vận tốc:

- ▶ $c_0 = (0, 0)$
- ▶ $c_{1,2,3,4} = (\pm 1, 0), (0, \pm 1)$
- ▶ $c_{5,6,7,8} = (\pm 1, \pm 1)$

Sự phụ thuộc xảy ra ở bước Streaming:

- ▶ Tại vị trí x , hàm phân bố f_i ở thời điểm tiếp theo phụ thuộc vào giá trị sau va chạm f_i^* từ điểm láng giềng.
- ▶ Cụ thể: $f_i(x, t + 1)$ nhận giá trị từ điểm $x - c_i \Delta t$

Xử lý trong chương trình:

Tuần tự

- ▶ Điều kiện biên Bounce-back tại tường (Y)
- ▶ Điều kiện biên Periodic tại biên X

Song song

- ▶ Cần trao đổi hàm phân bố biên giữa các CPU
- ▶ Sử dụng **Ghost Columns** (Cột ảo)

Ký hiệu:

- ▶ f_i : Hàm phân bố hạt theo hướng i ($i = 0 \dots 8$)
- ▶ c_i : Vector vận tốc rời rạc hướng i
- ▶ w_i : Trọng số lý thuyết
- ▶ $\omega = 1/\tau$: Tham số thư giãn

Đại lượng vĩ mô:

- ▶ Mật độ:

$$\rho = \sum_{i=0}^8 f_i$$

- ▶ Vận tốc:

$$u = \frac{1}{\rho} \sum_{i=0}^8 f_i c_i$$

Hàm phân bố cân bằng:

$$f_i^{eq} = w_i \rho \left(1 + 3(c_i \cdot u) + \frac{9}{2}(c_i \cdot u)^2 - \frac{3}{2}u^2 \right) \quad (2)$$

```
1 void CollisionCucBo(double *f, double *f_new, double *rho,
2                   double *ux, double *uy, int nx_local,
3                   int ny, double omega) {
4     for (int x = 0; x < nx_local; x++) {
5         for (int y = 0; y < ny; y++) {
6             double r = *(rho + x*ny + y);
7             double u_x = *(ux + x*ny + y);
8             double u_y = *(uy + x*ny + y);
9             double usqr = u_x*u_x + u_y*u_y;
10
11             for (int i = 0; i < Q; i++) {
12                 double cu = cx[i]*u_x + cy[i]*u_y;
13                 double feq = w[i] * r * (1.0 + 3.0*cu
14                                     + 4.5*cu*cu - 1.5*usqr);
15                 double fi = *(f + (x*ny + y)*Q + i);
16                 *(f_new + (x * ny + y) * Q + i) = fi - omega * (fi - feq);
17             }
18         }
19     }
20 }
```

Listing 1: Hàm Collision - Toán tử BGK

```
1 void StreamingCucBo(double *f_new, double *f, int nx_local,
2                   int ny, double *ghost_left,
3                   double *ghost_right, int rank, int size) {
4     for (int x = 0; x < nx_local; x++) {
5         for (int y = 0; y < ny; y++) {
6             for (int i = 0; i < Q; i++) {
7                 int xn = x + cx[i];
8                 int yn = y + cy[i];
9
10                if (yn < 0 || yn >= ny) { // Bounce-back
11                    *(f + (x*ny+y)*Q + opposite[i]) += *(f_new + (x*ny+y)*Q + i);
12                } else if (xn < 0) { // Ghost Left
13                    if (rank > 0) *(f + (x*ny+yn)*Q + i) += *(ghost_left + yn*Q + i);
14                } else if (xn >= nx_local) { // Ghost Right
15                    if (rank < size-1) *(f + (x*ny+yn)*Q + i) += *(ghost_right + yn*Q + i);
16                } else { // Trong m i n
17                    *(f + (xn*ny+yn)*Q + i) += *(f_new + (x*ny+y)*Q + i);
18                }
19            }
20        }
21    }
22 }
```

Listing 2: Hàm StreamingCucBo - Xử lý biên với Ghost

Mô hình: Single Program Multiple Data (SPMD)

Chia miền (Domain Decomposition):

- ▶ Chia 1D theo cột (chiều X)
- ▶ Mỗi tiến trình: $nx_local \approx NX/P$ cột
- ▶ Chiều Y: Không chia

Ví dụ: 256×64 với 4 tiến trình

- ▶ Rank 0: Cột 0-63
- ▶ Rank 1: Cột 64-127
- ▶ Rank 2: Cột 128-191
- ▶ Rank 3: Cột 192-255

Vùng đệm biên:

- ▶ Ghost Columns
- ▶ Kích thước: $NY \times Q$ doubles
- ▶ Với $NY = 64$: $64 \times 9 \times 8 = 4.5$ KB

Giao tiếp:

- ▶ Trao đổi với láng giềng trái/phải
- ▶ Sử dụng MPI non-blocking
- ▶ MPI_Isend / MPI_Irecv
- ▶ MPI_Waitall đồng bộ

5 Bước Chính trong Giải thuật

1. Khởi tạo:

- ▶ Cấp phát bộ nhớ: f , f_{new} , ρ , u , ghost buffers
- ▶ Đặt trạng thái ban đầu: $\rho = 1$, $u = 0$

2. Chia miền:

- ▶ Tính nx_local cho mỗi Rank
- ▶ Xử lý phần dư: $NX \bmod P$

3. Phân tán:

- ▶ Mỗi Rank tự khởi tạo vùng của mình
- ▶ Hoặc Rank 0 phát tán dữ liệu

4. Lặp tính toán: (10,000 bước)

- ▶ Tính đại lượng vĩ mô $\rightarrow$ Collision $\rightarrow$ Trao đổi biên $\rightarrow$ Streaming

5. Tổng hợp:

- ▶ Rank 0 thu thập kết quả (MPI_Reduce)
- ▶ Ghi file output



Sử dụng Non-blocking Communication:

- ▶ Tối ưu hóa thời gian chờ
- ▶ Cho phép tính toán và giao tiếp đồng thời

Quy trình trao đổi:

1. Rank i gửi cột 0 sang Rank $i - 1$, nhận vào `ghost_left`
2. Rank i gửi cột cuối sang Rank $i + 1$, nhận vào `ghost_right`
3. Sử dụng `MPI_Waitall` để đồng bộ

Quan trọng

Phải đợi tất cả giao tiếp hoàn thành trước khi thực hiện Streaming!

```
1 void TraoDoiGhost(double *f_new, int nx_local, int ny,  
2                 double *ghost_left, double *ghost_right,  
3                 int rank, int size, MPI_Comm comm) {  
4     MPI_Request reqs[4];  
5     int rcount = 0;  
6  
7     // Trao đổi với Rank trái  
8     if (rank > 0) {  
9         MPI_Irecv(ghost_left, ny*Q, MPI_DOUBLE,  
10                rank-1, 100, comm, &reqs[rcount++]);  
11        MPI_Isend(f_new, ny*Q, MPI_DOUBLE,  
12                rank-1, 101, comm, &reqs[rcount++]);  
13    }  
14  
15    // Trao đổi với Rank phải  
16    if (rank < size-1) {  
17        MPI_Irecv(ghost_right, ny*Q, MPI_DOUBLE,  
18                rank+1, 101, comm, &reqs[rcount++]);  
19        MPI_Isend(f_new + (nx_local-1)*ny*Q, ny*Q, MPI_DOUBLE,  
20                rank+1, 100, comm, &reqs[rcount++]);  
21    }  
22  
23    if (rcount > 0) MPI_Waitall(rcount, reqs, MPI_STATUSES_IGNORE);  
24 }
```

Listing 3: Hàm trao đổi Ghost Columns



Thông số: $\omega = 1.0$, $Steps = 10,000$, $\nu = 0.1667$

Kết quả đo đạc

CPU	Time (s)	MLUPS	Speedup	Eff.
1	1.802	90.94	1.00×	100%
2	1.365	120.00	1.32×	66%
4	0.738	222.13	2.44×	61%
8	0.450	364.09	4.00×	50%

*MLUPS: Million Lattice Updates Per Second

Nhận xét:

- ▶ Speedup tốt với 2-4 tiến trình
- ▶ Hiệu suất $> 60\%$ với 4 CPUs
- ▶ MLUPS tăng gần tuyến tính
- ▶ Overhead tăng với nhiều tiến trình

Nguyên nhân giảm hiệu suất:

- ▶ Giao tiếp MPI
- ▶ Đồng bộ hóa
- ▶ Cache miss

Xác thực độ chính xác:

- ▶ Vận tốc trung bình: Serial vs MPI
- ▶ Sai số: $< 10^{-14}$ (machine epsilon)
- ▶ Kết quả khớp hoàn toàn!

Trường vận tốc:

- ▶ Đạt trạng thái ổn định
- ▶ Profile parabol chuẩn
- ▶ Phù hợp lý thuyết Poiseuille

Bảo toàn khối lượng:

- ▶ $\rho_{total} = const$
- ▶ Sai số $< 0.01\%$



[Velocity Field Plot]

Trường vận tốc với profile Poiseuille

[Performance Charts]

Biểu đồ Speedup và Efficiency

Ưu điểm của LBM:

- ▶ Tính cục bộ cao
- ▶ Dễ song song hóa
- ▶ Xử lý biên phức tạp đơn giản
- ▶ Phù hợp với MPI/GPU

Hạn chế:

- ▶ Overhead truyền thông
- ▶ Hiệu suất giảm với lưới nhỏ
- ▶ Yêu cầu bộ nhớ lớn (9 hướng)

Hướng phát triển:

1. Chia miền 2D:

- ▶ Giảm overhead giao tiếp
- ▶ Tốt hơn với nhiều CPU

2. Overlapping:

- ▶ Tính toán trong khi giao tiếp
- ▶ Tăng hiệu suất 10-20%

3. GPU (CUDA):

- ▶ Speedup 50-100×
- ▶ Phù hợp cho lưới lớn

4. Hybrid MPI+OpenMP:

- ▶ Tận dụng đa nhân
- ▶ Giảm số message MPI

- [1] T. Krüger, H. Kusumaatmaja, A. Kuzmin, O. Shardt, G. Silva, and E. M. Viggien, *The Lattice Boltzmann Method: Principles and Practice*, Springer, 2017.
- [2] S. Chen and G. D. Doolen, “Lattice Boltzmann Method for Fluid Flows,” *Annual Review of Fluid Mechanics*, vol. 30, pp. 329–364, 1998.
- [3] S. Succi, *The Lattice Boltzmann Equation for Fluid Dynamics and Beyond*, Oxford University Press, 2001.
- [4] FlowKit Ltd., “Palabos: Parallel Lattice Boltzmann Solver,” <http://www.palabos.org/>, 2024.
- [5] “The Lattice Boltzmann Method,” <http://www.lbmethod.org/>, truy cập 2026.
- [6] Message Passing Interface Forum, “MPI: A Message-Passing Interface Standard,” <https://www.mpi-forum.org/>, Version 4.0, 2021.

THÔNG TIN LIÊN HỆ

Sinh viên: Krizpham

Môn học: IT5408 - Tính toán Hiệu năng Cao

Giảng viên: [Tên giảng viên]

Source code:

- ▶ Serial: `lbm_serial.c`
- ▶ MPI: `lbm_mpi.c`
- ▶ Benchmark: `benchmark.sh`
- ▶ Visualization: `visualize_results.py`

Xin chân thành cảm ơn Thầy và các bạn đã lắng nghe!

A decorative graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern composed of many small red dots. The dots are arranged in concentric, slightly offset rings, creating a sense of depth and movement. The word "HUST" is centered within this pattern.

HUST

THANK YOU !

